NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Project Report – Contact Directory

NU ID - Name:
24K-0527 – Muhammad Furqan Sheikh
24K-0826 – Muhammad Abu Bakar
24K-1020 – Fazeel Muhammad Mirza

—

Syllabus: Computer Organization & Assembly Language Lab

Submission Date: 11/24/2025

—

Instructor:

Sir Muhammad Owais

Contact Directory In Assembly Language

1. Executive Summary

This project uses the Irvine32 library and x86 Assembly Language to create a Contact Directory system. Creating a useful console application that functions as a digital phonebook was the primary objective. It enables users to add, edit, remove, and view contacts, among other common administrative functions. Additionally, we included sophisticated capabilities like alphabetising the list, looking for certain individuals, and colour-coding contacts according to "Friends" or "Favourites."

Important Findings The project effectively illustrates that high-level languages are not necessary for the development of complicated applications. Using low-level register logic, we were able to create a functional Binary Search and Bubble Sort. Learning how to manually manage data in memory, particularly using parallel arrays to store names, emails, and addresses, was a crucial lesson.

2. Introduction

The following project serves as a practical implementation of the concepts learned in our COAL course, that we were enrolled in the Fall 2025 semester at the National University for Computer and Emerging Sciences. While high-level languages handle details automatically, the project (as was made on a low-level language) requires us to interact directly with the system's hardware architecture. It is highly relevant to COAL because it forces us to manually manage computer memory and CPU registers.

By creating a contact directory, we are applying theoretical knowledge of the x86 processor architecture. We are defining exactly how many bytes of memory are reserved for data (Data Segment) and instructing the processor on how to move that data between memory and general-purpose registers (like EAX, ESI, and EDI). This project bridges the gap between software logic and the physical hardware, demonstrating how a computer executes instructions at machine level.

3. Project Description

Scope This was a console-based Management System.
The system features a Main Menu, Create-Read-Update-Delete (CRUD) functionalities, sorting logic, binary search, and auto-search. It also handles UI elements like changing text colours for different contact categories. The idea behind keeping the number of contacts fixed to a maximum of 20 contacts was to keep the memory management predictable.

Language: x86 Assembly (MASM).
Tools: Visual Studio (for coding and debugging).
Libraries: Irvine32
Data Handling: We used BYTE arrays for storing strings and WORD for counting contacts.

4. Methodology

Approach We didn't try to write the whole program in one go. Instead, we used a modular approach We broke the project down into small, manageable procedures.

1. First, we set up the .data section to ensure our arrays (names, nums, etc.) were defined correctly.
2. Next, we built the "Add" and "Display" features to test if data was actually being saved.
3. Finally, once the basics were working, we added the complex logic for Sorting and Searching.

5. Project Implementation

We structured the data using Parallel Arrays. Since we can't create a Class or Struct in Assembly easily, we created four separate arrays in memory:

- names
- nums
- addrs
- emails

These arrays share a common index. For example, the name stored at index 0 matches the phone number stored at index 0.

To access a specific contact, we calculate the memory address using the formula:
Base Address + (Index * Data Size).

Functionalities Developed Along The Way

- Data Management: We created procedures to write strings into the arrays (Add Contact) and overwrite them when removing a user (Delete Contact).
- Search & Sort: We implemented a Bubble Sort to alphabetize names before performing a Binary Search. This makes finding a contact much faster than checking them one by one.
- Visuals: We used SetTextColor to make the "Friends" category appear in a different color, making the UI easier to read.

Challenges Faced

- Memory Calculation: The hardest part was getting the memory offsets right. If we miscalculated the position of a name by just one byte, the program would print garbage symbols or crash. We resolved this by strictly defining sizes (e.g., Name Size = 30) and double-checking our math.
- String Comparison: In Python or C++, comparing two names is easy. In Assembly, we had to write a loop to compare strings character-by-character using the ESI and EDI registers.

6. Results

The completed application functions seamlessly. We can include a maximum of 20 contacts, and the software stops us from exceeding that number. The sorting function properly organizes names from A to Z, and the search tool effectively locates contact information.

The Primary Menu displaying all 9 choices.
The "Add Contact" interface showcasing data input.
The "Display" interface illustrating the categories in color codes.

Testing and Validation:
We conducted manual tests for the project:
Boundary Testing:
We attempted to add a 21st contact to verify that the program prevented us.
Search Testing:
We looked for a name that wasn't present to ensure the program managed the error smoothly without failing.
Functionality Test:
We inserted a contact, modified their information, and subsequently removed them to confirm that the entire process operated correctly without damaging the memory

7. Conclusion

The following project proved that we can build complex, logical applications using low-level instructions. Careful memory management's importance was highlighted. If it was important in such a project then for sure it would be very crucial in the complex application and algorithms that we see as of today's date. While Assembly code is much longer than C++ code for the same task, it gives us total control over the hardware. So, this is a trade-off that can influence a developers' decision from use case to use case. We successfully met the problem statement by delivering a working directory with all requested features.

Working on this project was challenging but it made us learn a lot. It bridged the gap between writing code and understanding how the processor executes it. The experience with the Irvine32 library and manual stack/register management has given us a much stronger grasp of computer architecture.